

Name – Jyotsna Kasibhotla

Roll no – 23102B0078

COMMUNITY HEALTH SURVEY SYSTEM

DevOps Mini-Project — Week 1

1. Problem Statement

Community health survey information is often collected through paper forms, spreadsheets, or disconnected records. This makes it difficult to maintain consistent records, search previous entries, update survey status, and generate a quick summary for decision-making.

The proposed Community Health Survey System will provide a single, small web-based application for recording, viewing, updating and searching survey records, supported by a simple role-based status workflow and a summary dashboard.

1.1 Real-Time Need / Pain Points

- Data duplication — the same information may be entered more than once.
- Slow retrieval — searching paper files or large spreadsheets is time-consuming.
- Update difficulty — corrections and status changes are hard to track.
- Limited visibility — supervisors lack an immediate summary of survey progress.
- Weak workflow control — no clear distinction between draft, submitted, verified and closed records.
- Testing risk — manual testing after every code change is repetitive and error-prone.

2. Target Users

User	Need	Example Action
Surveyor	Capture and update survey data	Create / edit a survey record
Administrator	Manage users and workflow	Verify status / manage records
Health Officer	Review progress and summaries	View dashboard / verified data
Community Respondent	Provide survey information	Supply health details to a surveyor

3. Stakeholder List

Stakeholder	Interest / Responsibility
Project team	Design, code, test, document and demonstrate the MVP.
Faculty / evaluator	Evaluate requirements coverage, DevOps workflow and technical demonstration.
Survey administrators	Need reliable record management and workflow control.
Health officers	Need quick summaries and verified records.

Surveyors	Need fast data entry and updates.
Community respondents	Provide accurate information for survey records.

4. Objectives

- Provide a simple interface to create, view, update and search community health survey records.
- Introduce a basic role-based workflow for survey status.
- Provide a dashboard showing useful summary counts.
- Store survey records in a structured relational database.
- Automate build and testing through Maven and Jenkins.
- Use Selenium to validate critical browser-based user journeys.
- Package and deploy the application using Docker.
- Use Ansible to automate target-environment configuration.

5. Constraints

- 15-week academic timeline; scope must remain small.
- Limited team size, budget and infrastructure.
- The application is an MVP, not a production-grade healthcare platform.
- Sensitive health information must not be real personal medical data — use dummy or anonymized test data only.
- Local development should work fully before introducing Jenkins, Docker and Ansible.
- The DevOps demonstration is more important than adding unnecessary application features.

6. Measurable Success Criteria

These are practical acceptance targets for the MVP, not real-world measured statistics.

Metric	Target
Core CRUD	Create, view and update records successfully.
Search	Search returns matching records for the chosen field.
Workflow	At least 3–4 defined statuses with role-based transitions.
Dashboard	Displays totals by status and useful summary counts.
Automated tests	3–5 critical Selenium journeys execute through Maven.
CI gate	A failing test prevents deployment in Jenkins.
Containerization	Application starts successfully from the versioned Docker image.
Provisioning	Ansible can configure a clean target and can be rerun without unnecessary changes.

7. Approved MVP Scope

7.1 In Scope

- Authentication — simple login with predefined roles.
- Survey record fields — ID, name, age, gender, location, health condition, survey date, status and creator.
- CRUD — create, view and update survey records.
- Search — search by survey ID, name or another simple field.
- Status workflow — DRAFT → SUBMITTED → VERIFIED → CLOSED.
- Dashboard — total records and counts by status.
- Testing — JUnit plus 3–5 Selenium WebDriver journeys.
- DevOps — Git/GitHub, Maven, Jenkins, Docker and Ansible.
- Documentation — architecture, setup, troubleshooting, limitations and future scope.

7.2 Out of Scope

- Real patient medical records or clinical diagnosis.
- Mobile application.
- Complex AI/ML prediction.
- Payment or insurance integration.
- Advanced analytics and GIS mapping.
- Multi-hospital enterprise integration.
- Kubernetes or cloud-native orchestration.
- Production-grade identity management such as OAuth2/SSO.

7.3 Recommended Data Model

Table	Important Fields
users	id, username, password, role
surveys	id, name, age, gender, location, health_condition, survey_date, status, created_by

8. Week 1 Deliverable Summary

Problem statement, stakeholder list, objectives, constraints and an approved MVP scope, as documented above, mark the completion of Week 1. This freezes the foundation on which Weeks 2–15 (Agile planning, Git/GitHub, feature development, Jenkins CI, Selenium testing, Docker containerization, Ansible provisioning and final release) will build.